

Production-grade End-to-End Data Platform

10 real-world project ideas with batch + streaming, analytics, warehousing, quality, governance, BI, and AI layers

Goal: choose a project that solves a real operational problem, has accessible data, and proves you can design a modern production data platform end to end.

Prepared for: Zakaria

Date: April 4, 2026

What is included: 10 precise project ideas, real-world use cases, suggested data sources, architecture stacks, step-by-step build paths, and doability scores

Doability % = a practical estimate of data access, streaming realism, engineering complexity, and how much hidden domain friction you are likely to hit.

How to use this document

- Pick one project with a real operator persona: city ops lead, transit dispatcher, security manager, trader, or supply-chain analyst.
- Use a bronze -> silver -> gold architecture. Bronze stores raw source truth, silver standardizes events/entities, gold exposes business metrics and dashboards.
- Ship both batch and streaming paths. Historical backfills train models and build context; live feeds power alerts, freshness checks, and real-time decision support.
- Treat governance and quality as first-class features: tests, lineage, SLAs, observability, and cataloging should be visible in the final demo.
- The AI layer should sit on top of trustworthy curated marts and documents, not replace them. Let the LLM explain, summarize, search, and assist.

Reference stack you can reuse across almost all 10 projects

Core ingestion	Python connectors + Airflow/Prefect for batch; Kafka or Redpanda for event streams
Storage	S3/MinIO for raw bronze + BigQuery / Snowflake / Postgres / ClickHouse depending on workload
Processing	dbt for analytics engineering; Spark Structured Streaming or Flink for real-time logic
Data quality	Great Expectations or Soda, plus dbt tests and freshness SLAs
Governance	DataHub or OpenMetadata + OpenLineage/Marquez
BI	Superset, Metabase, Grafana, or Power BI
AI layer	FastAPI + LangChain/LlamaIndex + pgvector/OpenSearch + curated prompt templates
DevOps	Docker Compose locally, Terraform/GitHub Actions in cloud, Prometheus/Grafana for monitoring

Shortlist by easiest starting point

- Best first build: CivicOps 311, BikePulse, or VulnOps Copilot.
- Best wow-factor with real-time depth: CryptoLens or SkyOps.
- Best balance of data engineering + AI + business value: RiskWire or DevPulse OSS.

10-project comparison matrix

#	Project	Industry	Best available data sources	Doability
1	BikePulse	Urban mobility / micromobility	Citi Bike trip history (batch), GBFS station status/information feeds (stream), NOAA/NWS weather and alerts, calendar/holiday features.	92%
2	TransitPulse	Public transit	GTFS static schedules (batch), GTFS-RT feeds for trip updates / vehicle positions / alerts (stream), weather, planned maintenance notices.	88%
3	CivicOps 311	Public sector / smart city	NYC 311 complaint history (daily batch + latest refresh), weather, holiday/calendar data, optional census demographics and asset layers.	95%
4	AirAware	Environmental health	Open-Meteo air-quality API and weather forecast/history, OpenAQ where helpful, census or CDC baseline vulnerability data, school/clinic locations.	90%
5	GridPulse	Energy / utilities	EIA grid monitor / EIA-930 operational data, weather forecasts/history, calendar effects, optional ISO/RTO price or outage feeds where public access exists.	78%
6	CryptoLens	Crypto / digital markets	Binance trades/order-book WebSocket streams, CoinGecko historical market data, optional news sentiment feeds, funding/open-interest from exchange APIs.	91%
7	VulnOps Copilot	Cybersecurity	CISA KEV catalog, NVD CVE API and feeds, GitHub Advisory Database, internal asset inventory mock data, package manifests/SBOM samples.	89%
8	RiskWire	Supply chain / geopolitical risk / procurement	GDELT events and document feeds, optional sanctions/watchlists, supplier master mock data, geography reference tables, weather/disaster overlays.	84%
9	SkyOps	Aviation / airport operations	OpenSky live aircraft states, FAA NAS status / airport status and delays, weather, optional airport reference data.	76%
10	DevPulse OSS	Developer tools / SaaS / open source	GitHub public Events API, GH Archive historical events, repository issues/PR APIs, release metadata, docs/changelog markdown for RAG.	87%

Interpret the score carefully: a 75%-80% project can still be excellent, but it usually has more feed friction, geo/time complexity, or domain-specific edge cases.

1. BikePulse - Real-time Bike-share Rebalancing and Station Health

Urban mobility / micromobility | Real business purpose: Solve stock-out and dock-full incidents before riders get frustrated, while also detecting broken stations and bad demand spikes.

Doability 92%

Why it matters	Solve stock-out and dock-full incidents before riders get frustrated, while also detecting broken stations and bad demand spikes.
Best data mix	Citi Bike trip history (batch), GBFS station status/information feeds (stream), NOAA/NWS weather and alerts, calendar/holiday features.
Recommended stack	Ingestion: Python + Kafka/Redpanda + Airflow. Storage: S3/MinIO bronze, BigQuery or Snowflake warehouse. Stream: Spark Structured Streaming or Flink. Modeling: dbt. Geo: H3/PostGIS. Quality: Great Expectations or Soda. Governance: DataHub/OpenMetadata + OpenLineage. BI: Superset/Looker Studio. AI: RAG over SOPs, station notes, and incident playbooks with pgvector.
Build steps	• Backfill 12-24 months of trip history and weather into bronze; partition by day and station. • Ingest GBFS station_status and station_information every 30-60 seconds into Kafka topics. • Build silver models for station snapshots, demand windows, outages, and weather-normalized station state. • Create gold marts: stock-out risk, dock-full risk, rebalancing queue, station-health SLA, operator productivity. • Train short-horizon demand and imbalance models by station and 15-minute interval. • Wire data-quality tests for null lat/long, duplicate station IDs, impossible bike counts, and late feeds. • Ship maps, operator queues, and an assistant that answers: Which stations need trucks now and why?
MVP output	Live station map, 30-minute risk forecast, truck routing queue, and weekly rebalancing KPI dashboard.
Main challenge	Need good geo joins and careful late-arriving snapshot handling, but the data itself is very accessible.

2. TransitPulse - Delay, ETA, and Service-disruption Command Center

Public transit | Real business purpose: Give riders and operations teams a single place to understand where delay minutes are forming, which lines are degrading, and how weather or incidents affect service reliability.

Doability 88%

Why it matters	Give riders and operations teams a single place to understand where delay minutes are forming, which lines are degrading, and how weather or incidents affect service reliability.
Best data mix	GTFS static schedules (batch), GTFS-RT feeds for trip updates / vehicle positions / alerts (stream), weather, planned maintenance notices.
Recommended stack	Ingestion: Python/Go connectors + Kafka. Storage: lakehouse on S3/Delta or warehouse in BigQuery. Stream: Flink for headway, bunching, and ETA drift. Modeling: dbt. Quality: Great Expectations. Governance: OpenMetadata. BI: Superset/Power BI. AI: retrieval assistant over alert history, disruption runbooks, and rider comms templates.
Build steps	• Choose one agency with strong feeds such as MTA, WMATA, or MBTA; land static GTFS nightly. • Stream vehicle positions, trip updates, and alerts; persist raw protobuf payloads for replay/debugging. • Create silver entities for trips, blocks, stops, vehicles, alerts, and observed-vs-scheduled arrivals. • Compute gold metrics: headway adherence, bunching rate, ETA error, delay propagation, missed-stop probability. • Add geospatial corridor views and route heatmaps showing where reliability breaks first. • Test freshness, schema drift, impossible speeds, and stop-sequence integrity before promoting records. • Expose dashboards plus a chat assistant for questions like: Which routes are failing this morning and what changed?
MVP output	Route reliability scorecards, delay propagation graph, station ETA accuracy dashboard, and incident Q&A.
Main challenge	Transit feeds vary by agency; protobuf handling and schedule-to-realtime matching add complexity.

3. CivicOps 311 - Complaint Triage, SLA-breach Risk, and Neighborhood Service Scorecards

Public sector / smart city | Real business purpose: Help city operations prioritize complaints that are likely to breach SLA, cluster similar issues, and explain which neighborhoods are systematically underserved.

Doability 95%

Why it matters	Help city operations prioritize complaints that are likely to breach SLA, cluster similar issues, and explain which neighborhoods are systematically underserved.
Best data mix	NYC 311 complaint history (daily batch + latest refresh), weather, holiday/calendar data, optional census demographics and asset layers.
Recommended stack	Ingestion: Airbyte or Python + Airflow. Storage: Postgres/PostGIS or BigQuery. Stream: Kafka for latest complaint refreshes and score updates. Transform: dbt. Quality: dbt tests + Great Expectations. Governance: DataHub/OpenMetadata. BI: Metabase/Superset. AI: LLM copilot for complaint summarization, clustering labels, and neighborhood brief generation.
Build steps	• Backfill multiple years of 311 complaints with geography, agency, complaint type, and response timestamps. • Ingest daily or near-fresh updates, normalize agency names, addresses, and complaint taxonomies. • Build silver models for complaint lifecycle, duplicate reports, spatial clusters, and response timelines. • Create gold marts: SLA risk, hotspot score, repeat-issue score, neighborhood service health, agency workload. • Train models for breach probability and repeat-complaint recurrence by issue type and area. • Apply quality rules for invalid geocodes, impossible close times, duplicate complaint IDs, and taxonomy drift. • Publish operational dashboards and a civic analyst assistant that drafts weekly neighborhood issue briefs.
MVP output	Heatmap of hotspots, breach-risk queue, agency scorecards, and weekly auto-generated service memo.
Main challenge	Low; the dataset is large but structured, stable, and perfect for analytics engineering.

4. AirAware - Air-quality, Weather, and Asthma-risk Alerting Platform

Environmental health | Real business purpose: Give schools, clinics, employers, or municipalities a live risk signal for pollution spikes, vulnerable neighborhoods, and likely bad respiratory days.

Doability 90%

Why it matters	Give schools, clinics, employers, or municipalities a live risk signal for pollution spikes, vulnerable neighborhoods, and likely bad respiratory days.
Best data mix	Open-Meteo air-quality API and weather forecast/history, OpenAQ where helpful, census or CDC baseline vulnerability data, school/clinic locations.
Recommended stack	Ingestion: Python + Airflow/Prefect. Storage: warehouse + PostGIS. Stream: Kafka or direct event bus for hourly forecasts/current conditions. Transform: dbt. Quality: Great Expectations/Soda. Governance: OpenMetadata. BI: Superset/Power BI. AI: assistant that explains why a risk alert fired and drafts human-readable guidance.
Build steps	• Collect hourly current conditions, forecasts, and history for target cities or catchment areas. • Join air pollutants, meteorology, and demographic vulnerability layers at neighborhood or H3 level. • Build silver tables for pollutant exposures, forecast error, sensor/source coverage, and local baselines. • Create gold marts: neighborhood risk index, school exposure window, clinic load-risk proxy, alert history. • Add threshold-based and model-based alerting for PM2.5, ozone, or combined respiratory stress. • Validate source freshness, unit consistency, missing hours, and outlier concentration values. • Launch dashboards and a copilot that answers: Which areas need an alert now, and what should we tell people?
MVP output	City map, rolling risk index, exposure alerts, and plain-English health guidance assistant.
Main challenge	Strongly doable; the hardest part is getting the alert logic right and validating domain thresholds.

5. GridPulse - Electricity Load, Forecast-error, and Grid-stress Monitor

Energy / utilities | Real business purpose: Surface where demand, generation mix, or forecast error is drifting in a way that signals price spikes, reliability stress, or renewable balancing problems.

Doability 78%

Why it matters	Surface where demand, generation mix, or forecast error is drifting in a way that signals price spikes, reliability stress, or renewable balancing problems.
Best data mix	EIA grid monitor / EIA-930 operational data, weather forecasts/history, calendar effects, optional ISO/RTO price or outage feeds where public access exists.
Recommended stack	Ingestion: Python + Airflow. Stream: Kafka/Redpanda for fresh hourly updates. Storage: lakehouse or warehouse. Stream compute: Spark Structured Streaming. Transform: dbt. Quality: Great Expectations. Governance: OpenMetadata + lineage. BI: Superset/Looker. AI: analyst assistant over grid events, operator notes, and anomaly explanations.
Build steps	• Register for EIA API access and backfill hourly load, demand forecast, net generation, interchange, and fuel mix. • Join weather, region, and holiday features; standardize time zones carefully. • Create silver entities for balancing authority, interval load, forecast, generation mix, and interchanges. • Build gold marts: forecast error, ramp-rate anomalies, renewable penetration, stress index, region scorecards. • Add anomaly detection for abnormal ramps, large forecast misses, and unusual interchange patterns. • Test freshness, unit consistency, timezone alignment, and gaps in hourly series before serving data. • Expose regional dashboards and an assistant for questions like: Why did ERCOT-like regions spike today?
MVP output	Regional load dashboard, forecast-error anomaly feed, and operator summary assistant.
Main challenge	Data is public, but timezone and balancing-authority semantics need care; live pricing feeds vary by region.

6. CryptoLens - Market Microstructure, Liquidity, and Anomaly Detection Platform

Crypto / digital markets | Real business purpose: Detect sudden order-flow shifts, abnormal spreads, liquidation risk, and sentiment-moving news fast enough for trading desks, market makers, or risk teams.

Doability 91%

Why it matters	Detect sudden order-flow shifts, abnormal spreads, liquidation risk, and sentiment-moving news fast enough for trading desks, market makers, or risk teams.
Best data mix	Binance trades/order-book WebSocket streams, CoinGecko historical market data, optional news sentiment feeds, funding/open-interest from exchange APIs.
Recommended stack	Ingestion: WebSocket consumers in Python/Rust -> Kafka. Storage: ClickHouse for tick data, S3 for raw archives, warehouse for aggregates. Stream: Flink or Materialize. Transform: dbt on warehouse marts. Quality: custom stream validations + Great Expectations. Governance: DataHub. BI: Grafana/Superset. AI: assistant over market events, exchange notices, and strategy playbooks.
Build steps	• Subscribe to a small basket first: BTCUSDT, ETHUSDT, one altcoin; persist raw book/trade messages. • Replay and compact streams into normalized events, best-bid-offer snapshots, and minute bars. • Backfill historical market context from CoinGecko for model training and regime labeling. • Build gold marts: spread, depth imbalance, realized volatility, anomaly score, liquidation-risk proxy. • Create real-time detectors for volatility bursts, book thinning, price dislocations, and exchange outages. • Test event ordering, duplicate trade IDs, stale books, and clock skew between feeds. • Deliver dashboards plus a copilot that explains what moved, when it changed, and whether it is unusual.
MVP output	Tick dashboard, anomaly feed, liquidity heatmap, and event explanation bot.
Main challenge	Very buildable with public feeds; the challenge is engineering discipline around high-frequency data volume.

7. VulnOps Copilot - Vulnerability Prioritization and Patch Analytics

Cybersecurity | Real business purpose: Security teams drown in CVEs. This project helps them prioritize what matters now based on known exploitation, package exposure, severity, and blast radius.

Doability 89%

Why it matters	Security teams drown in CVEs. This project helps them prioritize what matters now based on known exploitation, package exposure, severity, and blast radius.
Best data mix	CISA KEV catalog, NVD CVE API and feeds, GitHub Advisory Database, internal asset inventory mock data, package manifests/SBOM samples.
Recommended stack	Ingestion: Python + Airflow + Kafka for new advisories. Storage: Postgres/BigQuery + document store for raw advisories. Search/RAG: Elasticsearch/OpenSearch + pgvector. Transform: dbt. Quality: Great Expectations. Governance: DataHub. BI: Metabase/Superset. AI: remediation copilot over CVEs, advisories, patch notes, and runbooks.
Build steps	• Backfill CVEs and advisories; normalize CVE IDs, CWE, CVSS, affected packages, and vendor/product names. • Stream new KEV additions and advisory updates into a latest-risk topic. • Create silver models for advisory deduplication, package matching, exploit evidence, and asset exposure. • Build gold marts: patch priority queue, exposure summary, KEV coverage, MTTR, and team workload. • Join mock asset inventory or SBOM data to simulate a real organization and remediation workflow. • Test CVE key uniqueness, version range parsing, duplicate advisories, and stale enrichment records. • Ship dashboards and a security assistant that answers: What should we patch first this week and why?
MVP output	Priority patch queue, exploitable-now dashboard, and CVE explanation/remediation assistant.
Main challenge	Excellent portfolio project because the data is public and the business use case is instantly understandable.

8. RiskWire - Global News and Geopolitical Risk Intelligence for Supply Chains

Supply chain / geopolitical risk / procurement | Real business purpose: Procurement and operations teams need early warning when conflict, protest, sanctions, or disasters could hit suppliers, ports, routes, or critical regions.

Doability 84%

Why it matters	Procurement and operations teams need early warning when conflict, protest, sanctions, or disasters could hit suppliers, ports, routes, or critical regions.
Best data mix	GDELT events and document feeds, optional sanctions/watchlists, supplier master mock data, geography reference tables, weather/disaster overlays.
Recommended stack	Ingestion: Python + Kafka + Airflow. Storage: data lake + warehouse + search index. Stream: Flink/Spark for entity extraction and region scoring. Transform: dbt for conformed marts. Quality: Great Expectations. Governance: OpenMetadata. BI: Superset. AI: RAG assistant over event articles, watchlists, supplier records, and analyst notes.
Build steps	• Backfill a few months of GDELT events/doc records for selected countries, ports, and supplier regions. • Stream new events every 15 minutes; enrich with entity extraction, geocoding, and severity classification. • Create silver tables for articles, events, actors, locations, themes, and supplier/location relationships. • Build gold marts: supplier risk score, region volatility index, event timeline, exposure heatmap, alert queue. • Use embeddings or keyword/entity matching to connect news to suppliers, facilities, or trade lanes. • Test deduplication, geocoding coverage, false-positive topic matches, and article freshness. • Publish dashboards and an assistant that answers: Which suppliers are newly at risk today and what evidence supports that?
MVP output	Country/region risk board, supplier watchlist, event timeline, and sourcing-risk assistant.
Main challenge	Text enrichment adds complexity, but the live event feed is excellent for a modern data+AI portfolio.

9. SkyOps - Aviation Delay and Airport Congestion Early-warning Platform

Aviation / airport operations | Real business purpose: Detect congestion, ground-delay buildup, and weather-linked disruption early so airport, airline, or travel-ops teams can react faster.

Doability 76%

Why it matters	Detect congestion, ground-delay buildup, and weather-linked disruption early so airport, airline, or travel-ops teams can react faster.
Best data mix	OpenSky live aircraft states, FAA NAS status / airport status and delays, weather, optional airport reference data.
Recommended stack	Ingestion: Python/Go connectors + Kafka. Storage: ClickHouse or BigQuery + geospatial store. Stream: Flink/Spark. Transform: dbt. Quality: custom spatial/time-series checks + Great Expectations. Governance: OpenMetadata. BI: Grafana/Superset. AI: operations assistant over delay notices, weather bulletins, and airport disruption playbooks.
Build steps	• Start with a handful of major airports and ingest FAA airport status plus OpenSky position/state data. • Persist raw flight-state snapshots and airport delay notices for replay and debugging. • Build silver models for airport arrivals/departures, holding patterns, runway-area density proxies, and delay states. • Create gold marts: airport stress index, likely disruption cascade, weather-delay correlation, airline/route views. • Add anomaly logic for abnormal holding, sudden drops in departures, and prolonged ground-delay programs. • Validate timestamp consistency, missing state vectors, airport mapping errors, and geofence logic. • Expose dashboards and an assistant that explains: Which airports are degrading now and what is driving it?
MVP output	Airport congestion board, live map, delay-cause timeline, and disruption explainer assistant.
Main challenge	Very impressive project, but geo/time matching is harder and some live feeds have usage constraints.

10. DevPulse OSS - Open-source Product Health and Maintainer Copilot

Developer tools / SaaS / open source | Real business purpose:
Open-source teams need a real operational view of contributor activity, issue health, release quality, and user pain points instead of just stars and vanity metrics.

Doability 87%

Why it matters	Open-source teams need a real operational view of contributor activity, issue health, release quality, and user pain points instead of just stars and vanity metrics.
Best data mix	GitHub public Events API, GH Archive historical events, repository issues/PR APIs, release metadata, docs/changelog markdown for RAG.
Recommended stack	Ingestion: GitHub REST + GH Archive backfill + Kafka for public events. Storage: BigQuery/Postgres + object storage. Transform: dbt. Quality: dbt tests + Great Expectations. Governance: DataHub. BI: Metabase/Superset. AI: maintainer assistant over issues, PRs, docs, and release notes.
Build steps	• Select 1-3 active repositories with meaningful traffic and backfill event history from GH Archive. • Ingest current public events plus issue, PR, label, and release metadata on a schedule. • Create silver models for contributors, issues, PR lifecycle, release cadence, discussions, and response time. • Build gold marts: maintainer load, issue aging, contributor funnel, release health, bug hot spots, user pain themes. • Add semantic clustering and retrieval over issues/docs so maintainers can ask for recurring pain points. • Test event duplication, repo renames, issue-vs-PR distinctions, and missing timestamps. • Deliver dashboards and a copilot that answers: What is hurting adoption, and what should we fix next?
MVP output	Repo health dashboard, issue aging board, release scorecard, and maintainer Q&A assistant.
Main challenge	Strongly feasible and highly portfolio-friendly, especially if you choose a repo with active public history.

Current data-source availability snapshot (used to judge doability)

- Citi Bike publishes both trip history downloads and GBFS real-time feeds.
- The National Weather Service API provides forecasts, alerts, and observations.
- GTFS-Realtime is a mature open specification, and agencies such as MTA, WMATA, and MBTA publish developer-accessible feeds.
- NYC 311 complaint datasets are openly published and refreshed continuously or daily depending on view.
- Open-Meteo provides weather and air-quality APIs, with free non-commercial access and no API key for core use cases.
- EIA publishes grid-monitor and EIA-930 operational data; API registration is straightforward but still adds some friction.
- Binance offers official market WebSocket streams and CoinGecko offers historical market endpoints.
- CISA KEV, NVD, and GitHub Advisory Database provide strong public cybersecurity intelligence foundations.
- GDELT updates global event data every 15 minutes and is ideal for risk-monitoring platforms.
- OpenSky and FAA/NAS status data make aviation projects feasible, though geospatial matching and usage constraints raise complexity.
- GitHub public events and GH Archive make software-product analytics highly feasible.

My blunt recommendation

- If you want the safest portfolio win, build CivicOps 311 or BikePulse first.
- If you want something that gets recruiters to stop scrolling, build VulnOps Copilot or RiskWire.
- If you want to prove serious real-time engineering, build CryptoLens or SkyOps - but only if you are ready to handle streaming edge cases properly.